

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, MinMaxScaler

df_original = pd.read_csv('nba.csv')

print("First 5 rows of the dataset:")
display(df_original.head())
```

First 5 rows of the dataset:

	Unnamed: 0	name	gp	min	pts	fgm	fga	fg	3p_ma
0	0	Brandon Ingram	36	27.4	7.4	2.6	7.6	34.7	0.5
1	1	Andrew Harrison	35	26.9	7.2	2.0	6.7	29.6	0.7
2	2	JaKarr Sampson	74	15.3	5.2	2.0	4.7	42.2	0.4
3	3	Malik Sealy	58	11.6	5.7	2.3	5.5	42.6	0.1
4	4	Matt Geiger	48	11.5	4.5	1.6	3.0	52.4	0.0

5 rows × 22 columns

```
In [2]: df_original.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1340 entries, 0 to 1339
Data columns (total 22 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Unnamed: 0      1340 non-null  int64
1   name            1340 non-null  object
2   gp              1340 non-null  int64
3   min             1340 non-null  float64
4   pts             1340 non-null  float64
5   fgm             1340 non-null  float64
6   fga             1340 non-null  float64
7   fg              1340 non-null  float64
8   3p_made         1340 non-null  float64
9   3pa             1340 non-null  float64
10  3p              1340 non-null  float64
11  ftm             1340 non-null  float64
12  fta             1340 non-null  float64
13  ft              1340 non-null  float64
14  oreb            1340 non-null  float64
15  dreb            1340 non-null  float64
16  reb             1340 non-null  float64
17  ast             1340 non-null  float64
18  stl             1340 non-null  float64
19  blk             1340 non-null  float64
20  tov             1340 non-null  float64
21  target_5yrs    1340 non-null  int64
dtypes: float64(18), int64(3), object(1)
memory usage: 230.4+ KB
```

```
In [3]: df_original.describe()
```

Out[3]:

	Unnamed: 0	gp	min	pts	
count	1340.000000	1340.000000	1340.000000	1340.000000	1340.00
mean	669.500000	60.414179	17.624627	6.801493	2.62910
std	386.968991	17.433992	8.307964	4.357545	1.68351
min	0.000000	11.000000	3.100000	0.700000	0.30000
25%	334.750000	47.000000	10.875000	3.700000	1.40000
50%	669.500000	63.000000	16.100000	5.550000	2.10000
75%	1004.250000	77.000000	22.900000	8.800000	3.40000
max	1339.000000	82.000000	40.900000	28.200000	10.2000

8 rows × 21 columns

```
In [4]: df_original.duplicated().sum()
```

Out[4]: np.int64(0)

```
In [5]: df = df_original.copy()

df['target_5yrs'] = (df['gp'] >= 5).astype(int)

df = df.drop(columns=['name'])
df = df.drop(columns=['Unnamed: 0'])

print("DataFrame after defining target_5yrs and dropping 'name', and
'Unnamed: 0':")
display(df.head())
```

DataFrame after defining target_5yrs and dropping 'name', and
'Unnamed: 0':

	gp	min	pts	fgm	fga	fg	3p_made	3pa	3p	ftm
0	36	27.4	7.4	2.6	7.6	34.7	0.5	2.1	25.0	1.6
1	35	26.9	7.2	2.0	6.7	29.6	0.7	2.8	23.5	2.6
2	74	15.3	5.2	2.0	4.7	42.2	0.4	1.7	24.4	0.9
3	58	11.6	5.7	2.3	5.5	42.6	0.1	0.5	22.6	0.9
4	48	11.5	4.5	1.6	3.0	52.4	0.0	0.1	0.0	1.3

```
In [6]: missing_values = df.isnull().sum()
print("\nMissing values before handling:")
print(missing_values[missing_values > 0])

for column in df.columns:
    if df[column].dtype in ['int64', 'float64']:
        median_val = df[column].median()
        df[column].fillna(median_val, inplace=True)

print("\nMissing values after handling:")
print(df.isnull().sum()[df.isnull().sum() > 0])
```

```
Missing values before handling:
Series([], dtype: int64)
```

```
Missing values after handling:
Series([], dtype: int64)
```

```
C:\Users\LENOVO
```

```
THINKPAD\AppData\Local\Temp\ipykernel_20708\3503663785.py:8:
```

```
FutureWarning: A value is trying to be set on a copy of a
DataFrame or Series through chained assignment using an inplace
method.
```

```
The behavior will change in pandas 3.0. This inplace method
will never work because the intermediate object on which we are
setting values always behaves as a copy.
```

```
For example, when doing 'df[col].method(value, inplace=True)',
try using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace
on the original object.
```

```
df[column].fillna(median_val, inplace=True)
```

```
In [7]: df['points_per_minute'] = df['pts'] / df['min']

df['efficiency_rating'] = (df['pts'] + df['reb'] + df['ast'] + df['stl']
+ df['blk'] - \
                        ((df['fga'] - df['fgm']) + (df['fta'] -
df['ftm']) + df['tov']))) / df['gp']

print("\nDataFrame after adding engineered features (points_per_minute,
efficiency_rating):")
display(df[['points_per_minute', 'efficiency_rating']].head())
```

DataFrame after adding engineered features (points_per_minute, efficiency_rating):

	points_per_minute	efficiency_rating
0	0.270073	0.200000
1	0.267658	0.222857
2	0.339869	0.068919
3	0.491379	0.077586
4	0.391304	0.108333

```
In [8]: df = df.drop(columns=['gp'])

X_before_scaling = df.drop(columns=['target_5yrs'])
y_target = df['target_5yrs']

scaler = StandardScaler()

X_scaled_array = scaler.fit_transform(X_before_scaling)

df_scaled = pd.DataFrame(X_scaled_array,
                          columns=X_before_scaling.columns)

df_scaled['target_5yrs'] = y_target.reset_index(drop=True)

print("\nDataFrame after scaling numerical features:")
display(df_scaled.head())
```

DataFrame after scaling numerical features:

	min	pts	fgm	fga	fg	3p_made
0	1.177066	0.137401	-0.017294	0.477347	-1.543407	0.658041
1	1.116860	0.091486	-0.373816	0.226801	-2.374651	1.179494
2	-0.279912	-0.367659	-0.373816	-0.329969	-0.320991	0.397315
3	-0.725434	-0.252873	-0.195555	-0.107261	-0.255795	-0.384861
4	-0.737475	-0.528360	-0.611497	-0.803224	1.341496	-0.645581

5 rows × 21 columns

```
In [9]: y = df_scaled['target_5yrs']

X = df_scaled.drop(columns=['target_5yrs'])

features_to_remove_due_to_multicollinearity = [
    '3pa',
    'fgm',
    'fga',
    'fta',
    'dreb'
]

features_to_remove_due_to_multicollinearity = [f for f in
features_to_remove_due_to_multicollinearity if f in X.columns]

X_selected = X.drop(columns=features_to_remove_due_to_multicollinearity,
errors='ignore')

print("\nOriginal number of features:", X.shape[1])
print("Number of features after selection:", X_selected.shape[1])
print("\nSelected Features (first 5 rows):")
display(X_selected.head())
```

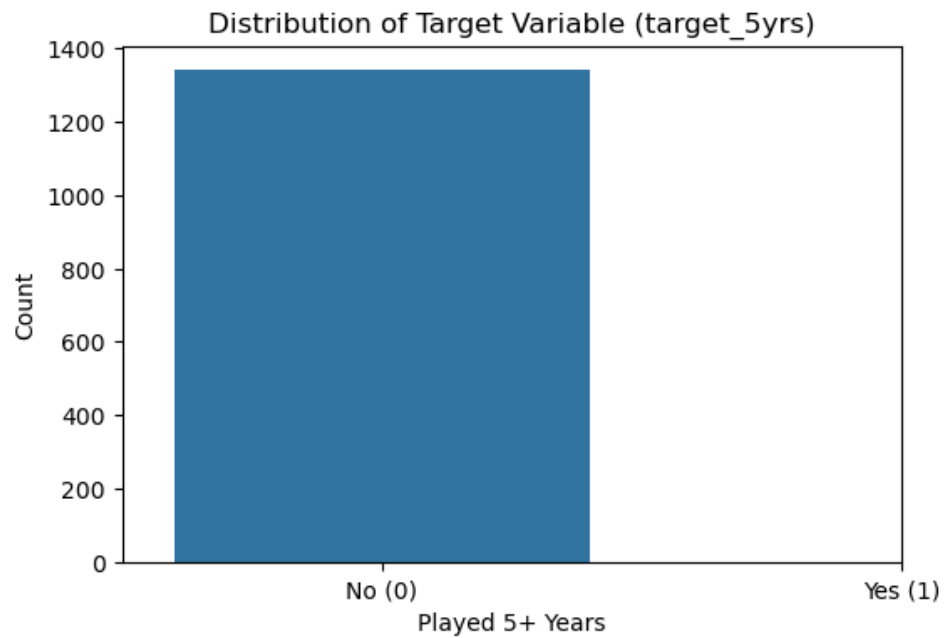
Original number of features: 20

Number of features after selection: 15

Selected Features (first 5 rows):

	min	pts	fg	3p_made	3p	ftn
0	1.177066	0.137401	-1.543407	0.658041	0.364603	0.306333
1	1.116860	0.091486	-2.374651	1.179494	0.271121	1.319630
2	-0.279912	-0.367659	-0.320991	0.397315	0.327210	-0.402971
3	-0.725434	-0.252873	-0.255795	-0.384863	0.215032	-0.402971
4	-0.737475	-0.528360	1.341496	-0.645589	-1.193430	0.002344


```
In [10]: plt.figure(figsize=(6, 4))
sns.countplot(x=y)
plt.title('Distribution of Target Variable (target_5yrs)')
plt.xlabel('Played 5+ Years')
plt.ylabel('Count')
plt.xticks(ticks=[0, 1], labels=['No (0)', 'Yes (1)'])
plt.show()
```

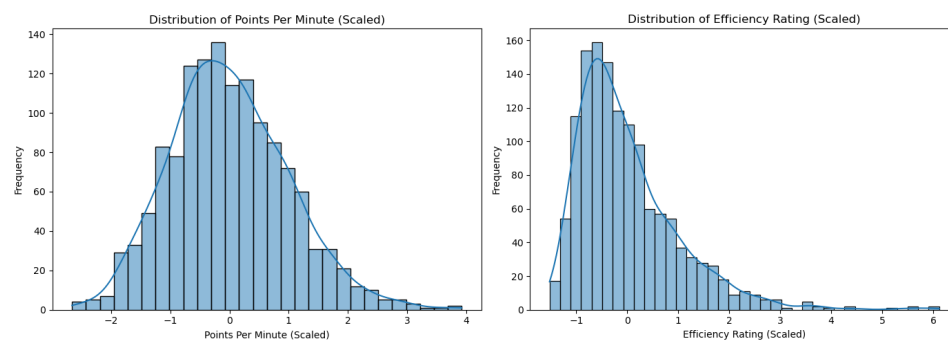


```
In [11]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.histplot(df_scaled['points_per_minute'], kde=True, ax=axes[0])
axes[0].set_title('Distribution of Points Per Minute (Scaled)')
axes[0].set_xlabel('Points Per Minute (Scaled)')
axes[0].set_ylabel('Frequency')

sns.histplot(df_scaled['efficiency_rating'], kde=True, ax=axes[1])
axes[1].set_title('Distribution of Efficiency Rating (Scaled)')
axes[1].set_xlabel('Efficiency Rating (Scaled)')
axes[1].set_ylabel('Frequency')

plt.tight_layout()
plt.show()
```



```

In [12]: correlation_matrix = X.corr()

threshold = 0.8

highly_correlated_pairs = []

for i in range(len(correlation_matrix.columns)):
    for j in range(i + 1, len(correlation_matrix.columns)):
        if abs(correlation_matrix.iloc[i, j]) > threshold:

highly_correlated_pairs.append((correlation_matrix.columns[i],
correlation_matrix.columns[j], correlation_matrix.iloc[i, j]))

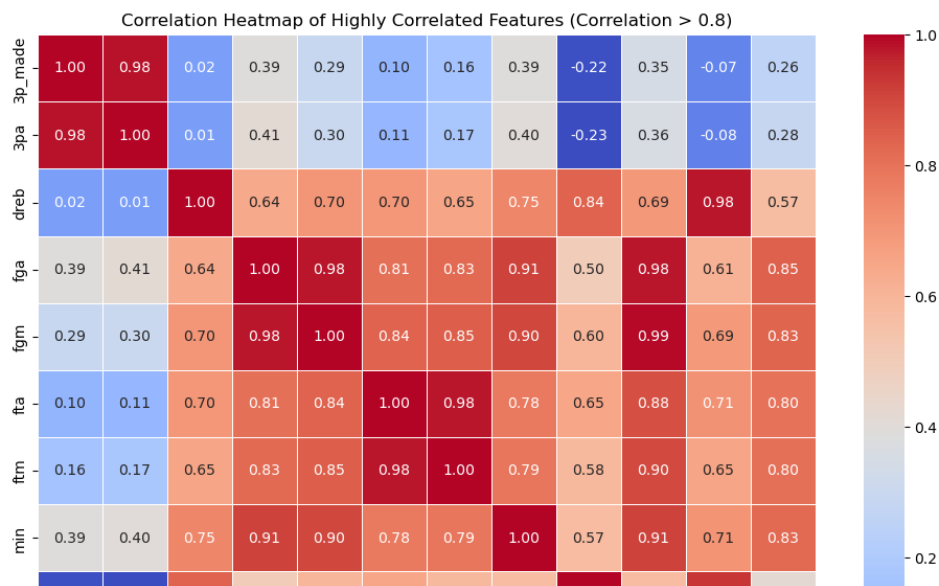
high_corr_features = set()
for col1, col2, _ in highly_correlated_pairs:
    high_corr_features.add(col1)
    high_corr_features.add(col2)

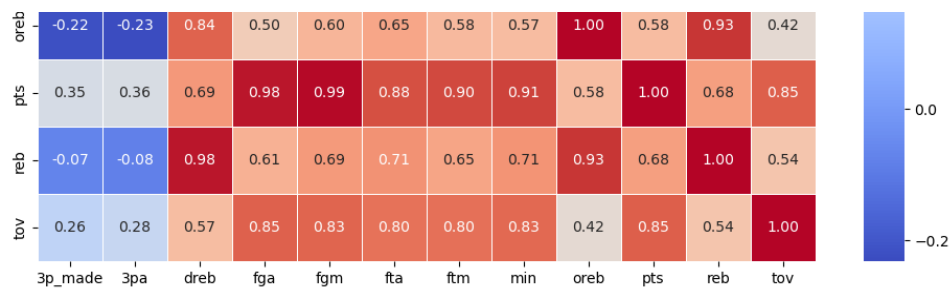
high_corr_features = sorted(list(high_corr_features))

if high_corr_features:
    filtered_correlation_matrix =
correlation_matrix.loc[high_corr_features, high_corr_features]

    plt.figure(figsize=(12, 10))
    sns.heatmap(filtered_correlation_matrix, annot=True,
cmap='coolwarm', fmt=".2f", linewidths=.5)
    plt.title('Correlation Heatmap of Highly Correlated Features
(Correlation > {}).format(threshold))
    plt.show()
else:
    print("No features found with correlation above the threshold of
{}".format(threshold))
    print("Consider lowering the threshold or checking the feature
set.")

```





In []:

In []: